

[Euron 10 주차] Tokenization & Embedding

1. Tokenization (토큰화)

1-1. NLP와 토큰화의 필요성

컴퓨터는 텍스트를 직접 이해하지 못하기 때문에 숫자 벡터로 변환해야 한다. 인간 언어는 모호성(다의적 해석), 가변성(다양한 표현), 구조성(문법·규칙)이라는 세 가지 특성을 가져 이 변환이 까다롭다. 토큰화는 이 변환의 첫 번째 단계다.

☀ 처리 흐름: 텍스트 입력 → 토큰화 → 수치화 → 모델 학습

1-2. 핵심 개념

- Corpus(말뭉치): 분석 대상 텍스트 데이터셋 (뉴스 기사, 리뷰 데이터 등)
- Token(토큰): 토큰화의 결과물인 개별 단위 (단어, 글자, 형태소)
- Tokenization(토큰화): 텍스트를 토큰으로 분리하는 과정
- Tokenizer(토큰나이저): 토큰화를 수행하는 규칙 또는 모델. 방식에 따라 결과가 크게 달라짐

1-3. 토큰나이저 구축 방법

- 공백 분할: 띄어쓰기 기준. 가장 단순하지만 한국어·중국어에 부적합
- 정규 표현식: 패턴 규칙으로 분리. 특수문자·URL 등 구조적 텍스트 처리에 유용
- 어휘 사전 기반: 등록된 단어만 인식. OOV(미등록 단어) 문제 발생 가능
- 머신러닝 기반: 데이터에서 규칙 자동 학습 (예: BPE, WordPiece)

1-4. OOV 문제

학습 시 어휘 사전에 없던 단어가 입력될 때 발생하는 문제. 어휘가 커질수록 Sparse data와 차원의 저주가 심화된다. 해결책으로 하위 단어 토큰화를 사용한다 — 모르는 단어를 아는 조각으로 분해해 처리한다.

1-5. 토큰화 단위 비교

- 단어 토큰화: 띄어쓰기 기준 분리. 직관적이고 의미 보존이 잘 되지만 OOV 가능성이 높다. 'cg'와 'cg.'가 다른 토큰으로 처리되는 문제도 발생한다.
- 글자 토큰화: 글자(자모) 단위 분리. 어휘 크기가 작고 OOV가 감소하지만 의미 정보가 부족하다. 한글은 초성·중성·종성으로 분리하는 자모 토큰화도 가능하다.
- 형태소 토큰화: 의미를 가진 최소 단위(자립·의존 형태소)로 분리. 문맥 이해가 가능하고 한국어(교착어)에 특화되어 있다. 품사 태깅(POS)과 함께 사용하면 더욱 정교한 분석이 가능하다.

1-6. 형태소 분석 라이브러리

- KoNLPy — Okt: SNS 텍스트 기반, 19개 품사. 빠른 처리에 적합
- KoNLPy — 꼬꼬마: 세종 말뭉치 기반, 56개 품사로 정밀 분석 가능. 다만 처리 속도가 느리고 모델 성능 저하 가능성이 있다
- NLTK: 영어 중심 라이브러리. Punkt 모델(통계적 문장 분리) + Averaged Perceptron Tagger(품사 태깅) 사용

1-7. 하위 단어 토큰화

기존 형태소 분석기는 신조어·오타자에 취약하고 어휘 사전이 비대해지는 문제가 있다. 하위 단어 토큰화는 하나의 단어를 빈번하게 등장하는 하위 단어 조합으로 나눠 이를 해결한다.

- BPE(Byte Pair Encoding): 가장 빈도 높은 글자 쌍을 반복 병합. 자주 등장하는 단어는 하나의 토큰으로, 드문 단어는 여러 조각의 조합으로 표현. 구글의 SentencePiece 라이브러리로 구현 가능
- WordPiece: BPE와 유사하나 빈도 대신 확률 기반으로 글자 쌍을 병합. $\text{Score} = f(x,y) / f(x) \cdot f(y)$ 수식으로 병합 적절성 판단. Hugging Face Tokenizers 라이브러리로 구현 가능

2. Embedding (임베딩)

2-1. 워드 임베딩

텍스트를 숫자 벡터로 변환하는 방법으로, 원-핫 인코딩·빈도 벡터화는 벡터의 희소성이 크다는 한계가 있다. 워드 임베딩은 비슷한 의미의 단어를 비슷한 벡터 공간에 위치시킨다 (예: Word2Vec, fastText). 단, 단어를 항상 고정된 값으로 표현하므로 문맥에 따라 의미가 달라지는 단어를 구분하기 어렵고, 이를 해결하기 위해 동적 임베딩(BERT, GPT 등)이 등장했다.

2-2. 언어 모델

- 언어 모델: 앞에 나온 단어들을 보고 다음에 올 단어를 예측하는 모델. 문장의 흐름과 문맥을 이해해 자연스러운 문장을 생성한다.
- 자기회귀 언어 모델: 이미 등장한 모든 단어를 참고해 다음 단어를 순서대로 예측. 조건부 확률의 연쇄법칙으로 표현된다.
- 통계적 언어 모델: 단어 순서·등장 빈도를 기반으로 예측. 주로 마르코프 체인으로 구현되며, 긴 문맥 처리와 미학습 단어 처리에 한계가 있다.
- 최근에는 대량의 텍스트를 사전 학습한 모델(GPT, BERT)이 주류다.

2-3. N-gram

연속된 N 개의 단어를 묶어 다음 단어를 예측하는 기본적인 언어 모델. 이전 N-1 개의 단어만 참고한다.

- N=1 유니그램, N=2 바이그램, N=3 트라이그램
- 적은 데이터에서도 문장 패턴 분석 가능. 관용 표현 분석에 활용
- Python 직접 구현 또는 `nltk.ngrams()` 함수로 구현

2-4. TF-IDF

문서에서 중요한 단어를 찾는 방법. 모든 단어를 동일 중요도로 취급하는 BoW(Bag-of-Words)의 한계를 보완한다.

- TF(단어 빈도): 특정 단어가 해당 문서에 얼마나 자주 등장하는지
- DF(문서 빈도): 특정 단어가 전체 문서 중 몇 개에 등장하는지
- IDF(역문서 빈도): 희귀 단어일수록 높은 값 부여. $\log(\text{전체 문서 수} / (1 + \text{DF}))$
- $\text{TF-IDF} = \text{TF} \times \text{IDF}$: 특정 문서에서 자주 등장하지만 전체 문서에서는 드문 단어를 중요하게 평가
- scikit-learn의 `TfidfVectorizer` 로 구현. `fit()`으로 말뭉치 학습, `transform()`으로 벡터 변환
- 한계: 단어의 순서·문맥을 고려하지 않으며, 벡터는 중요도를 나타낼 뿐 단어의 의미를 담지 않는다

3. 핵심 요약

NLP에서 텍스트는 토큰화 → 수치화 → 임베딩의 단계를 거쳐 모델이 처리할 수 있는 형태로 변환된다. 토큰화 방식은 단어·형태소·하위 단어 순으로 발전했으며, 벡터 표현은 TF-IDF(희소·중요도 기반)에서 워드 임베딩(밀집·의미 반영)을 거쳐 동적 임베딩(문맥 반영)으로 진화했다. 언어 모델 역시 N-gram 같은 통계 기반에서 BERT·GPT 같은 대규모 사전학습 모델로 발전하는 흐름을 따른다.